Do World Models Branch Where Physics Does? An Intuitive-Physics Test for Safe Manipulation*

Alikasim Budhwani **Soo Jeon**
Department of Mechanical and Mechatronics Engineering, University of Waterloo, Canada
a2budhwa@uwaterloo.ca

Abstract: Before a robot acts, it should know whether its own small errors could change the outcome. People judge this with an intuitive physics engine [1]: a few rough simulations over an uncertain scene, called precarious when the outcomes disagree. We build this check for robots as a *fork probe*: it samples the robot's measured perception and execution errors, rolls the intended action forward 64 times, and flags it when the futures split into two groups. With a physics engine and camera-based perception, the probe catches 88% of forks on unseen layouts with 7% false alarms. With a learned world model in place of the engine, average accuracy turns out not to be enough: a stochastic model that predicts 3–4× better than a no-motion baseline, with the right overall amount of spread, catches only 18% of forks, because its futures spread in every direction rather than along the branch. Deterministic dynamics started from perturbed scenes, as in the intuitive physics engine, reach 72% on a fresh test set, and 82% when training oversamples rare branches, found without labels. World models for safe deployment need this *branching fidelity*, which average-error benchmarks do not measure.

Keywords: World models, intuitive physics, uncertainty, safe deployment

1 Introduction

Manipulation policies act by committing to short action chunks [2, 3, 4]. Before each chunk runs, a safe robot should ask: could my own small errors change what happens? When a robot picks a block standing between two others, the same command can lift it cleanly or tip a neighbour, depending on a few millimetres of perception and execution error. Existing monitors flag inputs a model has not seen [5, 6]; this *physical* uncertainty remains even for a perfect model.

People judge it with an *intuitive physics engine*: a few rough simulations over an uncertain scene, whose disagreement signals precariousness [1, 7, 8]. We build the same check for robots. It needs a model of the robot's own uncertainty and a simulator; outside simulation, the simulator must be a learned world model. This turns the debate on whether robots need world models into a concrete test: do the model's rollouts split where the real physics splits? We find that they often do not, even when the model predicts well on average. We call the missing property *branching fidelity*.

Contributions. (1) A *fork probe*: a pre-execution check that pushes the robot's measured uncertainty through a simulator and flags actions whose futures split, with no task labels or tuned thresholds (Sec. 2). (2) With a physics engine and camera perception, it catches 88% of forks on unseen layouts, the bar for a world model (Sec. 3.1). (3) Learned world models: generative ones fail despite good average accuracy, deterministic ones come closer, and oversampling rare branches in training narrows the gap further (Sec. 3.2).

*A version of this work also appears at the CoRL 2026 Workshop on Bringing Physics Simulation and World Models Together for Robotic Manipulation.

CoRL 2026 Workshop "Do Robots Need World Models?", Austin, TX, USA.

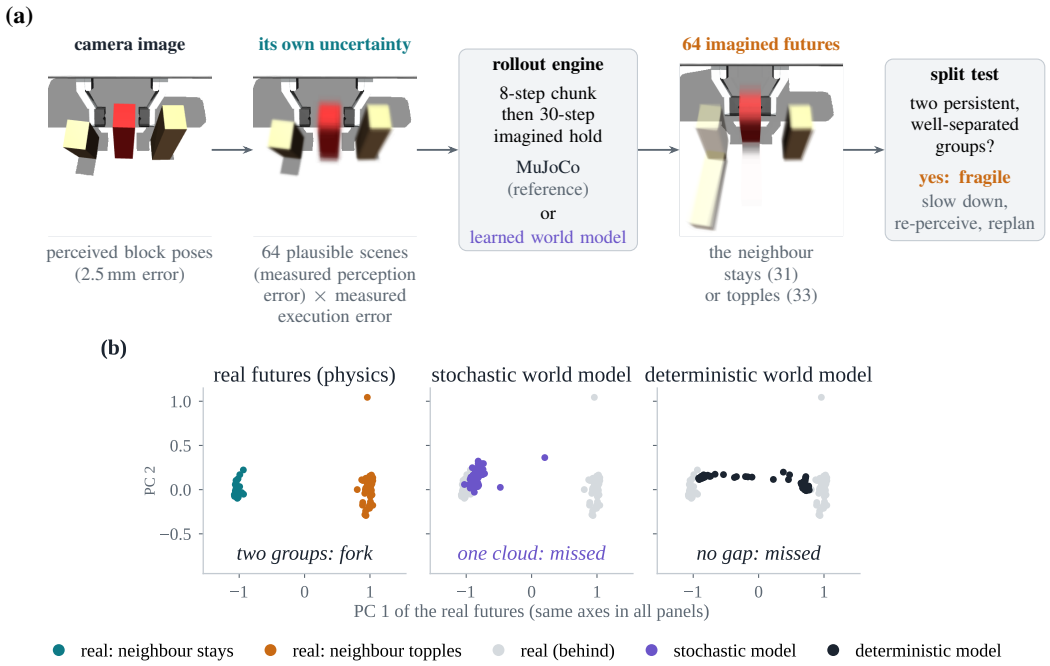


Figure 1: (a) The fork probe on one held-out state (exact renders; images average all 64 samples). (b) The 64 futures of that state, projected on the real futures' two main axes. The real futures form two groups (the neighbour stays or topples). The stochastic world model's futures form one cloud, so no fork is detected; the deterministic model's futures stretch along the branch but leave no gap.

2 The fork probe

Task. A simulated Franka Panda (MuJoCo) must pick the middle of three upright blocks without toppling its neighbours (Fig. 1a). A policy commits to 8-step action chunks; the probe treats it as a black box and sees only the chunk, one camera image and the robot's joint state.

Forks. The simulated physics is deterministic, so all uncertainty is the robot's own: its error in perceiving the block poses (2.5 mm, measured on held-out images) and in executing commands (about 3 mm, measured from tracking). A state is a *fork* if the outcome splits over 64 futures sampled from this uncertainty: here, if between 2 and 62 of them topple a neighbour. A fork is not a failure: every branch may be acceptable, and an action that always fails is not a fork. The probe asks only whether errors the robot cannot control decide the outcome; a task-level check is still needed (Sec. 5).

Probe. The probe imagines 64 versions of the chunk. Each starts from the perceived scene shifted by a sampled perception error and runs the chunk with a sampled execution error; the robot then holds still for 30 steps so the outcome settles. The probe flags the chunk if the 64 endings fall into two separate groups, a test of bimodality rather than of whether the endings differ, using fixed statistical rules instead of thresholds fit to labels (Appendix A). The simulator can be swapped: our reference is MuJoCo given the blocks' true shapes, masses and friction, which a real robot lacks; the alternative is a learned world model (Sec. 3.2).

Ground truth. One real outcome cannot grade the probe, since a fragile action may succeed by luck. An oracle runs the 64 futures from the *true* scene in exact physics and labels forks as above; two independent oracle runs agree on 99% of states. Each test set has unseen block layouts, about 60 forks and 100 safe states, and was fixed before testing.

Table 1: The fork probe with a physics engine on three unseen test sets: forks caught / false alarms (%).

	Set A (153 states)		Set B (160)		Set C (156)	
	caught	false	caught	false	caught	false
pose estimate trusted (no perception sampling)	65	23	65	12	67	24
fork probe, endings read from block positions	90	12	77	8	91	10
fork probe, endings read from images	88	12	77	8	**88**	**7**[*]

[*] with 5 sampled scene estimates (Appendix A); 9% with one.

Table 2: World models. Spread: total spread of the futures relative to real futures; main axis: its share along the real futures' main axis; fork ratio: spread on forks over spread on safe states. Fresh test sets were never used for training or selection.

rollouts from	caught	false	spread	main axis	fork ratio
test set C (156 states)					
physics engine (real futures)	88%	7%	1.00	0.57	4.8
world model, stochastic	18%	2%	0.98	0.15	1.3
world model, deterministic	60%	10%	0.04	0.69	32
two fresh test sets (160 / 152 states)					
uncertain starts + deterministic	72 / 82%	8 / 7%			
+ rare-branch oversampling	82 / 85%	9 / 11%			

3 Results

3.1 With a physics engine, the probe works

Sampling the perception error matters: a probe that trusts the pose estimate catches only 65–67% of forks, because perception errors are as large as execution errors; sampling them raises this to 77–91% (Table 1). Reading the endings from image features instead of block positions changes little: on test set C the probe catches 88% of forks (95% CI 77–94%) with 7% false alarms. This is the bar a learned world model must reach.

3.2 Learned world models: average accuracy is not branching fidelity

A learned model must now take MuJoCo's role: given one image and the 64 perturbed chunks, do its rollouts split where physics splits? Our models predict in a compact DINOv2 feature space [9] (Appendix A). Each training image comes with 8 real futures from perturbed starts, so a stochastic model can learn the robot's own uncertainty. We compare a stochastic model, which samples fresh noise at every step, with a deterministic one. Both predict 3–4$\times$ better than a no-motion baseline, yet on test set C the stochastic model catches 18% of forks and the deterministic one 60% (Table 2).

Why the stochastic model fails. It has the right *amount* of spread but the wrong *shape*: its futures spread equally in every direction, almost as much on safe states as on forks (Fig. 1b). The deterministic model's spread is tiny but lies along the branch and is 32$\times$ larger on forks. Five further generative variants catch at most 3% of forks, with the same shapeless spread (Appendix A).

Uncertain starts instead of noisy dynamics. As in the intuitive physics engine, uncertainty can enter through the start instead: a learned operator perturbs the start by sampled perception error, and a deterministic model rolls it out. Trained on fork-rich data, this catches 72% of forks on a fresh test set and plateaus there: longer training, more data, a frame history or DINO-WM's transformer predictor [10] all land at 68–72%. A generative model given the same perturbed starts still catches 0%, so the noise in its dynamics is what erases the forks.

Rebalancing rare branches. Toppling branches are rare in training and get averaged away. Without labels, we apply the split test to each training scene's own real futures, mark the smaller group as the rare branch, and fill half of every batch with rare-branch futures. On two fresh test sets this lifts the uncertain-start model from 72 to 82% and from 82 to 85% of forks: training can target branching fidelity, not only measure it.

4 Related work

Simulation under uncertainty. Robotics runs Monte Carlo rollouts over pose, friction or model parameters to score grasps and pushes [11, 12], to build safety filters [13, 14], or to judge whether blocks stay standing [15, 16, 17]; Goyal [18] measure offline how fast perturbed chunks diverge, and Zapolsky and Drumwright [19] find divergent traces to validate plans. These predict a task outcome, a stability label or a divergence rate. Our probe runs before every chunk, with measured errors and an automatic split test: a Monte Carlo test of closeness to a basin boundary [20].

Monitors and world models. Safety filters and failure detectors use failure labels, epistemic or policy uncertainty [21, 5, 6, 22]: what a model does not know, not whether the physics is at a branch. Learned world models [23, 24, 10, 25, 26] struggle with stiff contact [27] and show weak physical grounding in audits [28, 29, 30]. Yu et al. [31] evaluate branches created by different actions; ours arise from one action under small, unavoidable errors. Stochastic and ensemble models [32, 33] are judged by likelihood, calibration or return, none of which checks where their spread splits.

5 Conclusion

A robot can judge fragility as people do, by pushing its own measured uncertainty through a rough simulator and checking whether the futures split. A physics engine sets the bar at 88% of forks from one image. Learned world models fall short despite good average accuracy; deterministic dynamics with uncertain starts recover much of the gap, and rare-branch oversampling more, so benchmarks should measure branching fidelity beside average error. Because a fork is not a failure, a robot should combine the flag with a task- or harm-level signal before acting, for example by re-perceiving, slowing down or choosing a less fragile action. Whether an outcome can be undone is one label-free signal of harm; our follow-up work uses it as the failure set of a safety filter on a policy's actions. Next: pretrained world models such as V-JEPA 2-AC, and a real robot.

Limitations

- **Scope.** Simulation only, one task with three blocks; other tasks, real images and real robots are untested, and there is no closed-loop intervention or latency experiment here.
- **Ground truth.** The fork label uses the same perception and execution error model that the probe samples, so it measures agreement with that model, not with real outcomes; a real-robot anchor is needed. One Gaussian models perception error for all images, and the engine probe uses the simulator's true shapes, masses and friction. Sampled pose errors were applied without letting the blocks settle; a later check found that this moves a block with the robot still in 3 of 25 three-block states.
- **Split test.** It projects the endings on one main axis and fits at most two Gaussians, so it cannot represent more than two outcome regimes and can miss a split along a minor axis.
- **Operating points.** Each method is reported at one operating point fixed before testing; a sweep is possible only for the physics-engine probes on set C (Appendix B). The test sets differ in difficulty.
- **World models.** Each was trained once, at small scale, from scratch; the DINO-WM comparison re-implements its predictor on our latents rather than using the released model. The fork-rich training data of the uncertain-start models was screened with topple counts; only

the rare-branch oversampling is label-free, and the oversampled model also used a faster training recipe.

- **Forks are not failures.** Every branch of a fork may be acceptable, and an action that always fails is never a fork.

AI usage statement. Generative AI tools (Claude) assisted with code, figures, literature search and drafting. The authors designed the study, checked all results and references, and take full responsibility for the content.

References

[1] P. W. Battaglia, J. B. Hamrick, and J. B. Tenenbaum. Simulation as an engine of physical scene understanding. *Proceedings of the National Academy of Sciences*, 110(45):18327–18332, 2013.

[2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.

[3] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.

[4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv:2410.24164*, 2024.

[5] J. Seo, K. Nakamura, and A. Bajcsy. Uncertainty-aware latent safety filters for avoiding out-of-distribution failures. In *Conference on Robot Learning*, 2025.

[6] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems*, 2025.

[7] K. A. Smith and E. Vul. Sources of uncertainty in intuitive physics. *Topics in Cognitive Science*, 5(1):185–199, 2013.

[8] A. N. Sanborn, V. K. Mansinghka, and T. L. Griffiths. Reconciling intuitive physics and Newtonian mechanics for colliding objects. *Psychological Review*, 120(2):411–437, 2013.

[9] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[10] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, 2025.

[11] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio Ojea, and K. Goldberg. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. In *Robotics: Science and Systems*, 2017.

[12] W. C. Agboh and M. R. Dogar. Pushing fast and slow: Task-adaptive planning for non-prehensile manipulation under uncertainty. In *Workshop on the Algorithmic Foundations of Robotics (WAFR)*, pages 160–176, 2018.

[13] A. Johansson, D. Lindmark, V. Wiberg, and M. Servin. Safety filtering of robotic manipulation under environment uncertainty: a computational approach. *arXiv:2509.12674*, 2025.

[14] G. Wang, K. Ren, A. S. Morgan, and K. Hang. ManiDreams: An open-source library for robust object manipulation via uncertainty-aware task-specific intuitive physics. *arXiv:2603.18336*, 2026.

[15] R. Cannizzaro, M. Groom, J. Routley, R. O. Ness, and L. Kunze. COBRA-PPM: A causal Bayesian reasoning architecture using probabilistic programming for robot manipulation under uncertainty. *arXiv:2403.14488*, 2024.

[16] L. Zhou, K. A. Smith, J. B. Tenenbaum, and T. Gerstenberg. Mental Jenga: A counterfactual simulation model of causal judgments about physical support. *Journal of Experimental Psychology: General*, 152(8):2237–2269, 2023.

[17] A. Lerer, S. Gross, and R. Fergus. Learning physical intuition of block towers by example. In *International Conference on Machine Learning*, 2016.

[18] A. Goyal. Measuring the stability assumption behind action chunking. *arXiv:2610.01626*, 2026.

[19] S. Zapolsky and E. Drumwright. Particle traces for detecting divergent robot behavior. *arXiv:1608.01606*, 2016.

[20] P. J. Menck, J. Heitzig, N. Marwan, and J. Kurths. How basin stability complements the linear-stability paradigm. *Nature Physics*, 9:89–92, 2013.

[21] K. Nakamura, L. Peters, and A. Bajcsy. Generalizing safety beyond collision-avoidance via latent-space reachability analysis. In *Robotics: Science and Systems*, 2025.

[22] C. Agia, R. Sinha, J. Yang, Z.-a. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning*, 2024.

[23] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning*, 2020.

[24] K. R. Allen, Y. Rubanova, T. Lopez-Guevara, W. Whitney, A. Sanchez-Gonzalez, P. Battaglia, and T. Pfaff. Learning rigid dynamics with face interaction graph networks. In *International Conference on Learning Representations*, 2023.

[25] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*, 2025.

[26] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

[27] M. Parmar, M. Halm, and M. Posa. Fundamental challenges in deep learning for stiff contact dynamics. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.

[28] D. M. Bear, E. Wang, D. Mrowca, F. J. Binder, H.-Y. F. Tung, R. T. Pramod, C. Holdaway, S. Tao, K. Smith, F.-Y. Sun, L. Fei-Fei, N. Kanwisher, J. B. Tenenbaum, D. L. K. Yamins, and J. E. Fan. Physion: Evaluating physical prediction from vision in humans and machines. In *NeurIPS Datasets and Benchmarks*, 2021.

[29] S. Motamed, L. Culp, K. Swersky, P. Jaini, and R. Geirhos. Do generative video models understand physical principles? In *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026.

[30] T. Yang, Z. Shen, Z. Mi, Z. Zhang, J. Zhou, J. Ji, J. Dai, J. Chen, B. Chen, and Y. Yang. MiraBench: Evaluating action-conditioned reliability in robotic world models. *arXiv:2605.29360*, 2026.

[31] Y. Yu, S. Zhang, Y. Sheng, H. Ren, and H. Lin. How should world models be evaluated for embodied decision-making? A decision-making-centric position. *arXiv:2606.15032*, 2026.

[32] K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

[33] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, 2017.

[34] K. A. Ashman, C. M. Bird, and S. E. Zepf. Detecting bimodality in astronomical datasets. *The Astronomical Journal*, 108:2348–2361, 1994.

A Implementation details

Uncertainty model. Execution error is drawn from 64 snippets of the robot's measured tracking residuals (median per-chunk maximum 3.2 mm). Perception is a frozen DINOv2 encoder on one close-camera image with a small head that predicts the three blocks' poses (2.5 mm held-out error); its error model is a zero-mean Gaussian fitted once, offline, to its errors on held-out images with known poses (a full 9×9 position covariance capturing correlated errors, plus rotation covariances).

Actions and the hold. An action is an absolute end-effector position target plus a gripper command, tracked by an impedance controller for 0.1 s. Chunk step k executes the commanded target plus the sampled execution error; during the 3 s hold the arm holds the chunk's last nominal target, without execution error. The world models receive exactly the same action sequence, with no special hold input.

Split test. The 64 endings (block positions, or DINOv2 features of the rendered endings) are projected on their first principal axis at hold steps 10 and 30. The chunk is flagged if, at both times, two Gaussians fit better than one (BIC), the groups are well separated (Ashman's $D > 2$ [34]) and the smaller group has at least 2 members, and the grouping is the same at both times. These are fixed statistical conventions, not fitted to labels.

Five scene hypotheses. Each hypothesis draws one scene estimate from the perception-error model around the perceived scene; its 64 futures add a fresh perception draw and an execution-error sample each. The probe flags if at least 3 of the 5 hypotheses split.

Ground truth. The oracle is the probe centred on the true scene: each of its 64 futures starts from the true block poses shifted by a fresh perception-error draw and uses one execution-error sample (the same 64 snippets as the probe, different perception seeds). A neighbour topples if it tilts more than 45° by the end. Test-set states were found by an execution-only screen, not by the oracle.

World models. The latent compresses DINOv2 patch tokens by a fixed per-patch PCA to $28 \times 28 \times 8$; applied to real futures, the split test on this latent still catches 97% of forks. The dynamics are a convolutional network with six residual blocks conditioned on the action. One model step covers two simulator actions (0.2 s), so the 8-step chunk plus the 30-step hold is 19 model steps; the quoted prediction error is the open-loop latent error at the end of this horizon (0.055 stochastic, 0.042 deterministic, 0.176 for a no-motion baseline). The stochastic model is a flow-matching transition that draws fresh noise at every step; the deterministic one predicts the next latent directly. Each training image comes with 8 real futures whose starts were perturbed by sampled perception error.

Generative variants. Whole-clip generation, an energy-score loss, 4× more fork data, and an action-sensitivity loss. On 160 held-out states from the training domain (75 forks) they catch 0–3% of forks at 0–1% false alarms, with spread 1.1–1.6× that of the real futures and only 7–10% of it on the main axis.

Uncertain starts. A learned operator perturbs the start latent by a sampled perception error and the deterministic network rolls each perturbed start out. On the 160 training-domain states it catches 52% of forks at 7% false alarms, and 69% at 8% with 4× more fork-rich data; there, the perturbations were paired with the real futures' own draws. On the fresh test sets every draw is independent, as in deployment. Variants with 3× longer training, more fork data, a 3-frame history or DINO-WM's transformer predictor [10] reach 68–72% on the training-domain states.

Rare-branch oversampling. For every training state, the split test is applied to the latents of its real futures; if it splits, the smaller group at hold step 30 is the rare branch. Each batch draws half its futures from rare branches and half from the other branch of the same starts. Model and loss are unchanged, but this model was trained with a faster recipe (larger batches, the same number of samples); a model without oversampling was not retrained with that recipe. The fresh test sets have 160 states (67 forks) and 152 states (55 forks), with reset seeds disjoint from training but chunks from the same 100 demonstrations; neither was used for training or model selection. The first had earlier served as a held-out check for other models; the second was generated for this check.

B Operating-point sweep

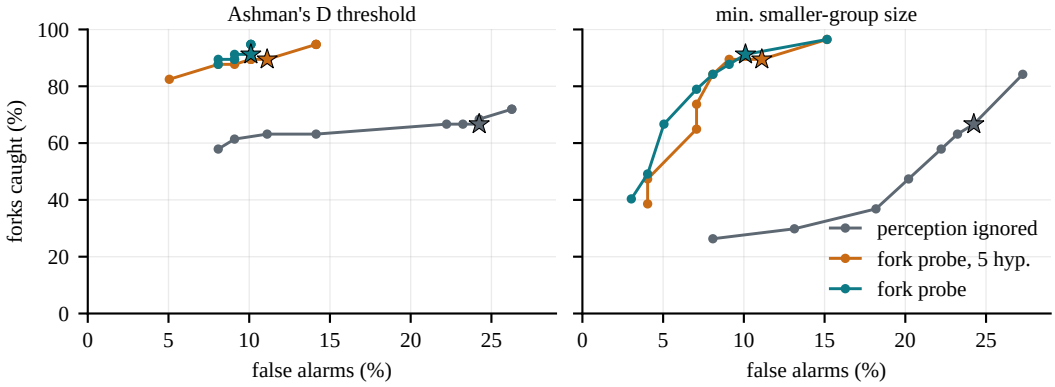


Figure 2: Forks caught vs false alarms on test set C (57 forks, 99 safe states) as one split-test setting is varied with the others at their defaults; stars mark the defaults used in Table 1. Left: Ashman's D threshold (0.5–8). Right: minimum size of the smaller group (1–16).

Recomputed from the stored set-C rollouts, the defaults reproduce Table 1 exactly. Across every Ashman's-D threshold from 0.5 to 8, the fork probe catches 88–95% of forks at 8–10% false alarms, while the probe that ignores perception uncertainty never catches more than 72%, and only 58% once its false alarms fall to 8%. Results barely move between $D = 1.5$ and 4, so the default is not a tuned optimum. The minimum group size is the most sensitive setting: requiring at least 8 members per group halves every method's catch rate. With five scene hypotheses, requiring 2, 3 or 4 of the 5 to split gives 95/21, 89/11 and 84/7% (caught / false); adding a margin to the BIC comparison changes little up to +10. The probes' advantage holds at every operating point. Rollouts for sets A and B and for the world models were not stored, so this sweep covers the physics-engine probes on set C only.